

**Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники**

**Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение
Дисциплина «Вычислительная математика»**

Лабораторная работа №6

Вариант 7

Выполнила

Матвеева Полина Павловна

Проверила

Наумова Надежда Александровна

Санкт-Петербург, 2026

Цель работы

Решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.

Описание алгоритма решения задачи

Алгоритм работы программы состоит из следующих этапов: ввод исходных данных, проверка их корректности, вычисление решения выбранными методами, оценка точности и вывод результатов.

1. **Выбор дифференциального уравнения.** Пользователь выбирает одно из предложенных уравнений вида $y' = f(x, y)$ из списка, реализованного в программе (не менее трёх уравнений).
2. **Ввод исходных данных.** Пользователь вводит:
 - начальные условия $x_0, y_0 = y(x_0)$;
 - границы интервала интегрирования $[x_0, x_n]$;
 - шаг интегрирования h ;
 - требуемую точность ε .
3. **Проверка корректности данных.** Программа проверяет введённые значения:
 - все числовые поля заполнены и являются числами;
 - $x_n > x_0$ (правая граница больше левой);
 - шаг $h > 0$ и не превышает длину интервала $(x_n - x_0)$;
 - точность $\varepsilon > 0$;
 - для уравнений с особыми точками (например, содержащих y/x) проверяется, что интервал $[x_0, x_n]$ не содержит точку разрыва.

При обнаружении некорректных данных вывод соответствующего сообщения об ошибке и повторный запрос ввода.
4. **Выбор методов решения.** Пользователь отмечает, какие методы необходимо применить: одношаговые (метод Эйлера, усовершенствованный метод Эйлера) и/или многошаговый метод (метод Милна). Если ни один метод не выбран – вывод сообщения об ошибке.
5. **Построение расчётной сетки.** На основе x_0, x_n и h строится равномерная сетка узлов $x_i = x_0 + i \cdot h, i = 0, 1, \dots, n$, где $n = \lceil (x_n - x_0)/h \rceil$.
6. **Вычисление приближённого решения.** Для каждого выбранного метода вычисляется таблица значений $y_i \approx y(x_i)$:
 - для одношаговых методов – последовательно по рекуррентной формуле метода, начиная с y_0 ;
 - для метода Милна – первые три точки находятся методом Рунге–Кутты 4-го порядка (разгон), далее применяется схема предиктор–корректор с итерационным уточнением корректора до выполнения условия $|y^{(k+1)} - y^{(k)}| < \varepsilon$.

7. **Вычисление точного решения.** Поскольку для всех уравнений известна аналитическая формула $y(x)$, вычисляется эталонная таблица значений $y_i^{\text{точн}} = y(x_i)$ для сравнения с приближённым решением.

8. **Оценка точности методов.**

- для одношаговых методов – повторное вычисление решения с шагом $h/2$ и применение правила Рунге: $R_i = \frac{|y_i^{(h)} - y_i^{(h/2)}|}{2^p - 1}$, где p – порядок метода;
- для метода Милна – непосредственное сравнение с точным решением: $\varepsilon_{\text{Милн}} = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i|$.

Полученная оценка сравнивается с заданной точностью ε ; делается вывод о достижении требуемой точности.

9. **Вывод результатов.**

- таблица значений x_i , точного решения и приближённых решений всех выбранных методов;
- численные оценки точности для каждого метода;
- график точного решения и приближённых решений (разными цветами для каждого метода).

10. **Анализ результатов.** На основании таблиц и графиков делается сравнение методов по точности и по поведению погрешности при изменении шага h (формулируется в выводах к работе).

Рабочие формулы методов

Метод Эйлера

Одношаговый метод первого порядка точности ($p = 1$). Решение строится по формуле

$$y_{i+1} = y_i + h \cdot f(x_i, y_i), \quad i = 0, 1, \dots, n-1.$$

Оценка погрешности по правилу Рунге:

$$R_i = \frac{|y_i^{(h)} - y_i^{(h/2)}|}{2^p - 1} = |y_i^{(h)} - y_i^{(h/2)}|.$$

Усовершенствованный метод Эйлера (метод Хойна)

Одношаговый метод второго порядка точности ($p = 2$). На каждом шаге сначала вычисляется прогноз, затем поправка:

$$\begin{aligned} \tilde{y}_{i+1} &= y_i + h \cdot f(x_i, y_i), \\ y_{i+1} &= y_i + \frac{h}{2} \left(f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1}) \right). \end{aligned}$$

Оценка погрешности по правилу Рунге:

$$R_i = \frac{|y_i^{(h)} - y_i^{(h/2)}|}{2^p - 1} = \frac{|y_i^{(h)} - y_i^{(h/2)}|}{3}.$$

Метод Рунге—Кутты 4-го порядка (для разгона метода Милна)

Используется для вычисления первых трёх точек, необходимых для запуска многошагового метода Милна:

$$\begin{aligned} k_1 &= h \cdot f(x_i, y_i), \\ k_2 &= h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right), \\ k_3 &= h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), \\ k_4 &= h \cdot f(x_i + h, y_i + k_3), \\ y_{i+1} &= y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Метод Милна

Четырёхшаговый метод четвёртого порядка точности ($p = 4$), реализованный по схеме предиктор – корректор. Используются четыре предыдущих точки $(x_{i-4}, \dots, x_{i-1})$ и значения правой части $f_j = f(x_j, y_j)$.

Предиктор:

$$y_i^{\text{пр}} = y_{i-4} + \frac{4h}{3}(2f_{i-3} - f_{i-2} + 2f_{i-1}).$$

Корректор (применяется итерационно до достижения заданной точности ε):

$$y_i^{\text{кор}} = y_{i-2} + \frac{h}{3}(f_{i-2} + 4f_{i-1} + f(x_i, y_i^{\text{кор}})).$$

Итерации по корректору повторяются до выполнения условия

$$\left| y_i^{\text{кор},(k+1)} - y_i^{\text{кор},(k)} \right| < \varepsilon.$$

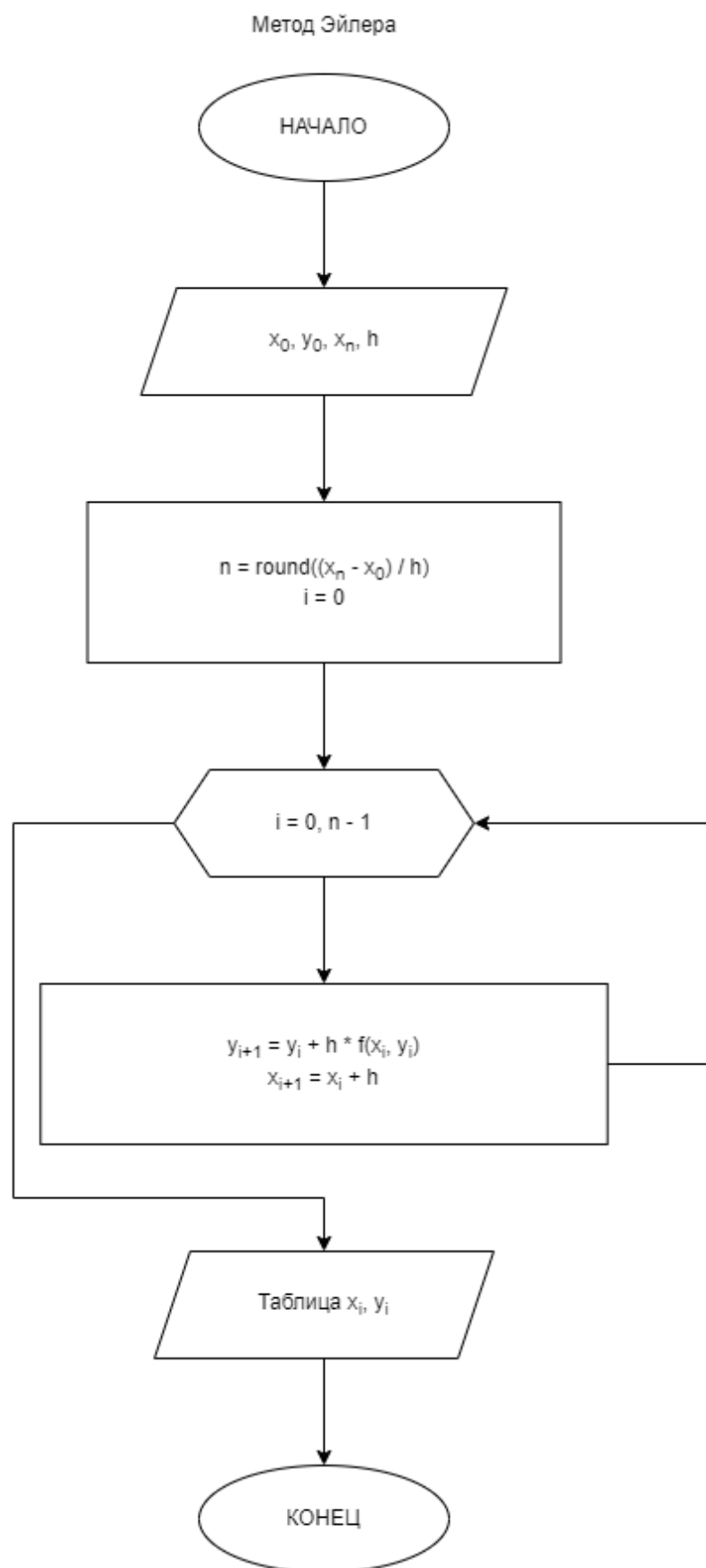
Оценка точности метода Милна производится не по правилу Рунге, а непосредственно сравнением с точным решением:

$$\varepsilon = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i|.$$

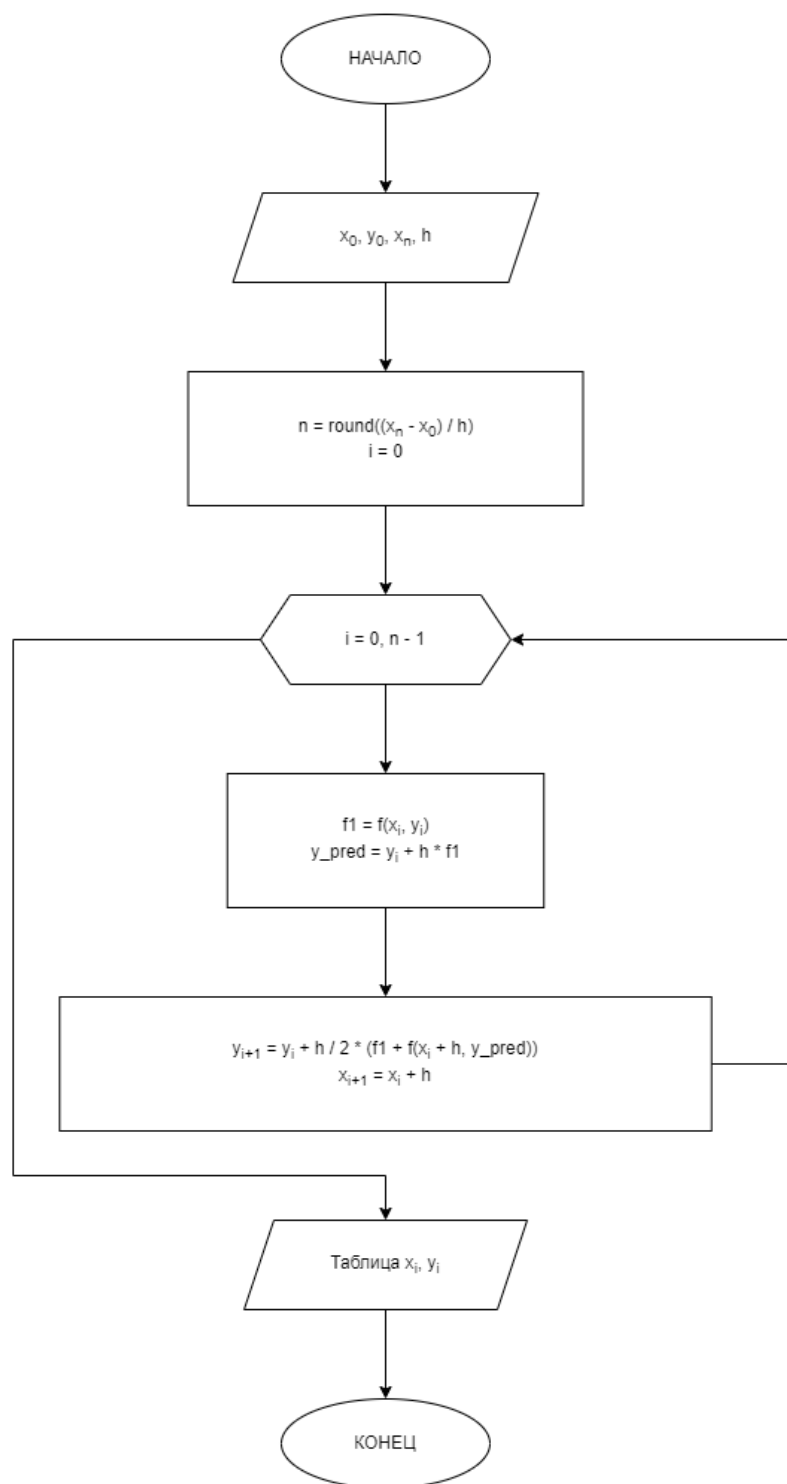
Точное (аналитическое) решение

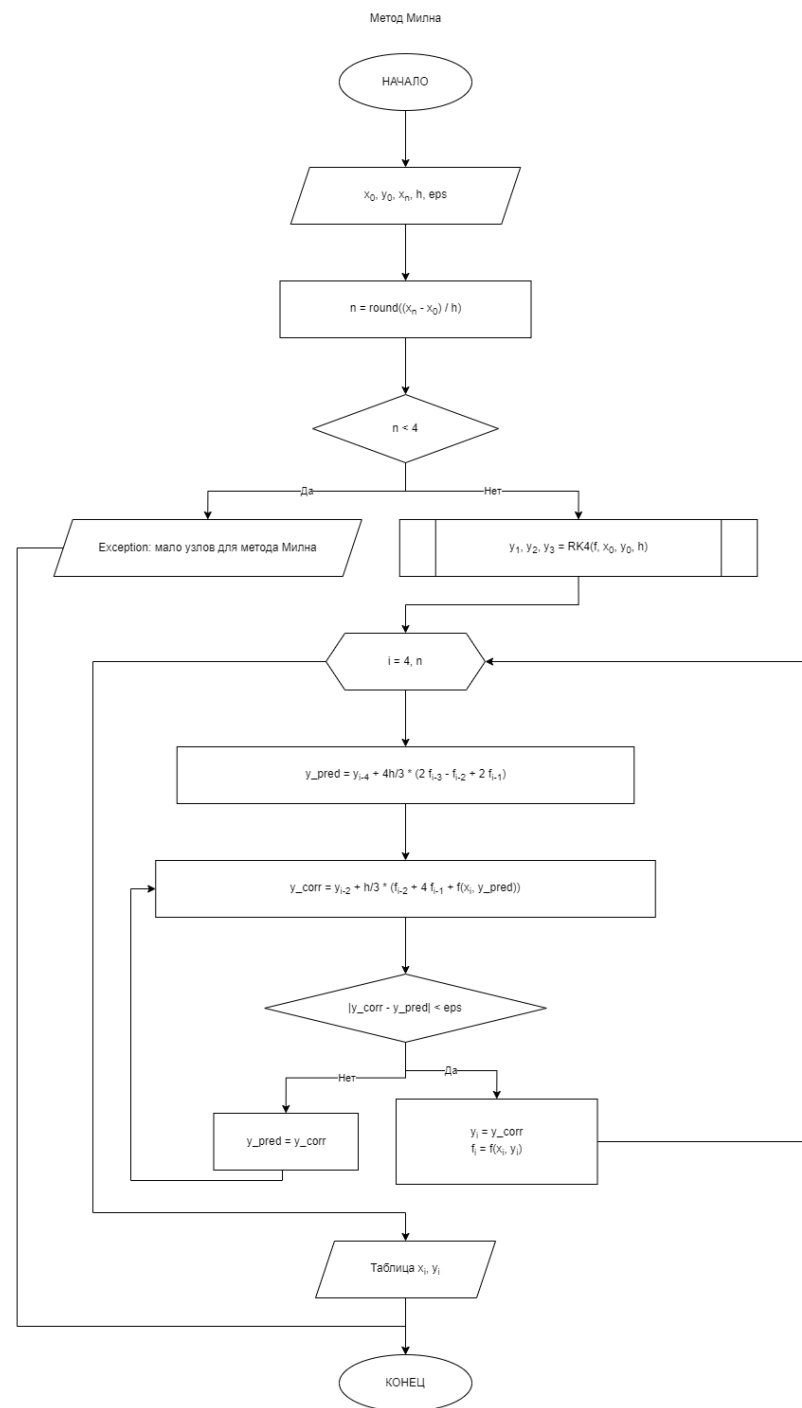
Для каждого из рассматриваемых уравнений $y' = f(x, y)$ точное решение $y(x)$ находится аналитически (методом интегрирующего множителя, поскольку все уравнения линейны относительно y) и используется как эталон при оценке погрешности численных методов.

Блоксхемы



Усовершенствованный метод Эйлера





Листинг методов

```

1
2 def euler(f, x0, y0, xn, h):
3     xs = make_grid(x0, xn, h)
4     ys = [y0]
5     for i in range(len(xs) - 1):
6         ys.append(ys[i] + h * f(xs[i], ys[i]))
7     return xs, ys
8
9
10 def improved_euler(f, x0, y0, xn, h):
11     xs = make_grid(x0, xn, h)
12     ys = [y0]
13     for i in range(len(xs) - 1):

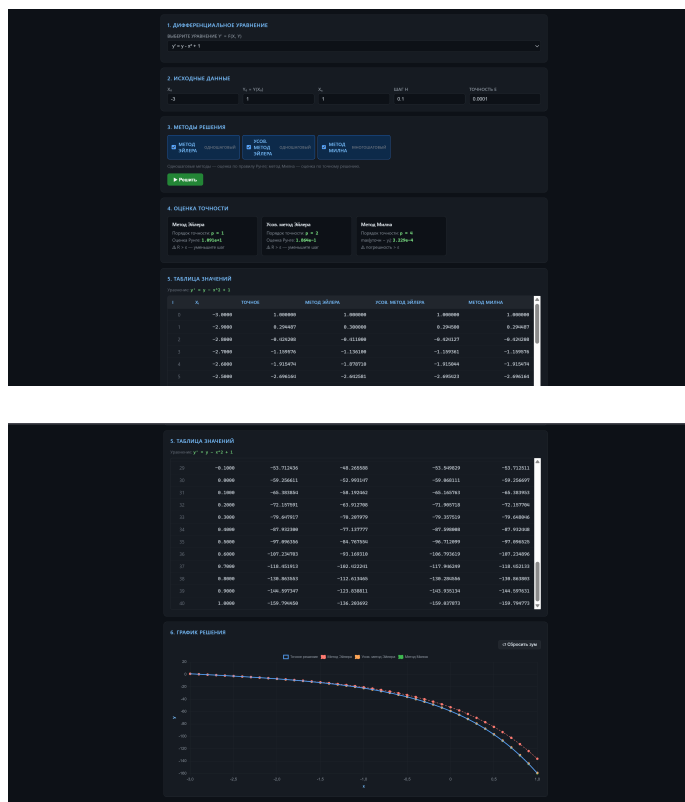
```

```
14         fi = f(xs[i], ys[i])
15         y_predict = ys[i] + h * fi
16         ys.append(ys[i] + h / 2.0 * (fi + f(xs[i] + h, y_predict)))
17     return xs, ys
18
19
20 def _rk4_step(f, x, y, h):
21     k1 = h * f(x, y)
22     k2 = h * f(x + h / 2.0, y + k1 / 2.0)
23     k3 = h * f(x + h / 2.0, y + k2 / 2.0)
24     k4 = h * f(x + h, y + k3)
25     return y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
26
27
28 def milne(f, x0, y0, xn, h, eps):
29     xs = make_grid(x0, xn, h)
30     ys = [y0]
31     fs = [f(xs[0], ys[0])]
32
33     start = min(3, len(xs) - 1)
34     for i in range(start):
35         y_next = _rk4_step(f, xs[i], ys[i], h)
36         ys.append(y_next)
37         fs.append(f(xs[i + 1], y_next))
38
39     for i in range(4, len(xs)):
40         y_pred = ys[i - 4] + 4.0 * h / 3.0 * (2.0 * fs[i - 3] - fs[i - 2]
+ 2.0 * fs[i - 1])
41         f_pred = f(xs[i], y_pred)
42         y_corr = ys[i - 2] + h / 3.0 * (fs[i - 2] + 4.0 * fs[i - 1] +
f_pred)
43         for _ in range(50):
44             f_corr = f(xs[i], y_corr)
45             y_new = ys[i - 2] + h / 3.0 * (fs[i - 2] + 4.0 * fs[i - 1] +
f_corr)
46             if abs(y_new - y_corr) < eps:
47                 y_corr = y_new
48                 break
49             y_corr = y_new
50         ys.append(y_corr)
51         fs.append(f(xs[i], y_corr))
52     return xs, ys
53
54
55 def runge_error(solver, f, x0, y0, xn, h, order):
56     xs_h, ys_h = solver(f, x0, y0, xn, h)
57     _, ys_half = solver(f, x0, y0, xn, h / 2.0)
58     denom = 2.0 ** order - 1.0
59     errors = [abs(ys_h[i] - ys_half[2 * i]) / denom for i in range(len(
xs_h))]
60     return errors, max(errors)
61
62
63 def exact_solution(equation_key, xs, x0, y0):
64     exact = EQUATIONS[equation_key]['exact']
65     return [exact(x, x0, y0) for x in xs]
66
67
68 def max_exact_error(ys, ys_exact):
69     return max(abs(ya - ye) for ya, ye in zip(ys, ys_exact))
```

Программная реализация

Исходный код: ссылка на полную реализацию

Скриншоты программы



Выводы

В работе реализована программа для численного решения ОДУ первого порядка тремя методами: Эйлера, усовершенствованным методом Эйлера и методом Милна. Точность одношаговых методов оценивалась по правилу Рунге, многошагового – сравнением с точным решением. Расчёты подтвердили ожидаемое поведение методов: метод Эйлера наименее точен, усовершенствованный метод Эйлера точнее при тех же затратах, метод Милна даёт наибольшую точность за счёт более высокого порядка. Программа корректно обрабатывает некорректные входные данные.